INFORME DE LABORATORIO

(formato estudiante)

INFORMACIÓN BÁSICA					
ASIGNATURA:	INTRODUCCIÓN AL DESARROLLO DE NUEVAS PLATAFORMAS (E)				
TÍTULO DE LA PRÁCTICA:	Pantallas múltiples con NavigationBar y comunicación entre Composables en Jetpack Compose				
NÚMERO DE PRÁCTICA:	5	**AÑO LECTIVO:**	2026-B	**NRO. SEMESTRE:**	VIII
FECHA DE PRESENTACIÓN	03/09/2026	**HORA DE PRESENTACIÓN**	23:59		
INTEGRANTE (s): - Zuñiga Villacorta, Peter Sebastian				**NOTA (0-20)**	
DOCENTE(s): Roxana Evelyn Limache Calatayud					

RESULTADOS Y PRUEBAS

Github Repo: https://github.com/UNSA-IS-2026-B/idnp_lab/tree/main/lab05

1. EJERCICIOS RESUELTOS

A continuación se detallan, paso a paso, las instrucciones para construir una aplicación con tres pantallas (Home, Edificios y Mapa) navegables desde una barra inferior, comunicando la selección de un edificio mediante una función lambda.

Paso 1: Crear un nuevo proyecto y definir una sealed class Screen con tres objetos: Home, Edificios y Mapa, cada uno con su propiedad route (String) y label (String).

```kotlin
sealed class Screen(val route: String, val label: String) {
    object Home : Screen("home", "Home")
    object Edificios : Screen("edificios", "Edificios")
    object Mapa : Screen("mapa", "Mapa")
}
```

Paso 2: Verificar que el proyecto tenga las dependencias necesarias: abrir build.gradle.kts (Module :app) y confirmar que el bloque dependencies { ... } incluya implementation("androidx.navigation:navigation-compose:2.8.0") (para NavHost/NavController) e implementation("androidx.compose.material:material-icons-extended") (para los íconos Icons.Default.Home/List/Place de la barra de navegación); si falta alguna, agregarla y sincronizar el proyecto (botón “Sync Now”) antes de continuar.

Kotlin

```
dependencies {
    // Navigation Compose
    implementation("androidx.navigation:navigation-compose:2.8.0")

    // Iconos extendidos de Material 3
    implementation("androidx.compose.material:material-icons-extended")

    // ... demas dependencias por defecto ...
}
```

Paso 3: En el Composable raíz, crear val navController = rememberNavController() y una variable de estado var edificioSeleccionado by remember { mutableStateOf("Ninguno") }.

Paso 4: Envolver la UI en un Scaffold(bottomBar = { ... }). Dentro de bottomBar, declarar una NavigationBar con un NavigationBarItem por cada Screen, marcando como seleccionado el que coincide con la ruta actual (currentBackStackEntryAsState()) y llamando a navController.navigate(screen.route) al pulsarlo.

Paso 5: Dentro del padding entregado por el Scaffold, declarar un NavHost(navController, startDestination = Screen.Home.route) con un bloque composable(route) { ... } para cada una de las tres pantallas.

Kotlin

```
// Paso 3, 4 y 5: Composable Raíz con Scaffold, NavigationBar y NavHost
@Composable
fun MainScreen() {
    val navController = rememberNavController()
    var edificioSeleccionado by remember { mutableStateOf("Ninguno") }
    val items = listOf(Screen.Home, Screen.Edificios, Screen.Mapa)

    Scaffold(
        bottomBar = {
            NavigationBar {
                val navBackStackEntry by navController.currentBackStackEntryAsState()
                val currentRoute = navBackStackEntry?.destination?.route

                items.forEach { screen ->
                    val icon = when (screen) {
                        Screen.Home -> Icons.Default.Home
                        Screen.Edificios -> Icons.Default.List
                        Screen.Mapa -> Icons.Default.Place
                    }
                    NavigationBarItem(
                        selected = currentRoute == screen.route,
                        onClick = { navController.navigate(screen.route) },
                        icon = { Icon(icon, contentDescription = screen.label) },
                        label = { Text(screen.label) }
                    )
```

```
                }
            }
        }
    ) { padding ->
        NavHost(
            navController = navController,
            startDestination = Screen.Home.route,
            modifier = Modifier.padding(padding)
        ) {
            composable(Screen.Home.route) {
                HomeScreen(edificioSeleccionado = edificioSeleccionado)
            }
            composable(Screen.Edificios.route) {
                EdificiosScreen(
                    onEdificioSeleccionado = { nombre ->
                        edificioSeleccionado = nombre
                    }
                )
            }
            composable(Screen.Mapa.route) {
                MapaScreen()
            }
        }
    }
}
```

Paso 6: Implementar HomeScreen(edificioSeleccionado: String) mostrando un Text de bienvenida y otro Text con el valor recibido (por ejemplo "Último edificio consultado: $edificioSeleccionado").

Kotlin

```kotlin
// Paso 6: HomeScreen
@Composable
fun HomeScreen(edificioSeleccionado: String) {
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(text = "Bienvenido a la app de Edificios")
        Text(text = "Último edificio consultado: $edificioSeleccionado")
    }
}
```

Paso 7: Implementar EdificiosScreen(onEdificioSeleccionado: (String) -> Unit) con una LazyColumn que liste al menos 4 edificaciones (nombre + botón); al pulsar un ítem, invocar onEdificioSeleccionado(nombre).

```kotlin
// Paso 7: EdificiosScreen
@Composable
fun EdificiosScreen(onEdificioSeleccionado: (String) -> Unit) {
    val edificios = listOf("Biblioteca Central", "Pabellón A", "Pabellón B", "Auditorio")

    LazyColumn(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp)
    ) {
        items(edificios) { nombre ->
            Row(
                modifier = Modifier
                    .fillMaxWidth()
                    .padding(vertical = 8.dp),
                verticalAlignment = Alignment.CenterVertically
            ) {
                Text(text = nombre, modifier = Modifier.weight(1f))
                Button(onClick = { onEdificioSeleccionado(nombre) }) {
                    Text("Ver")
                }
            }
        }
    }
}
```

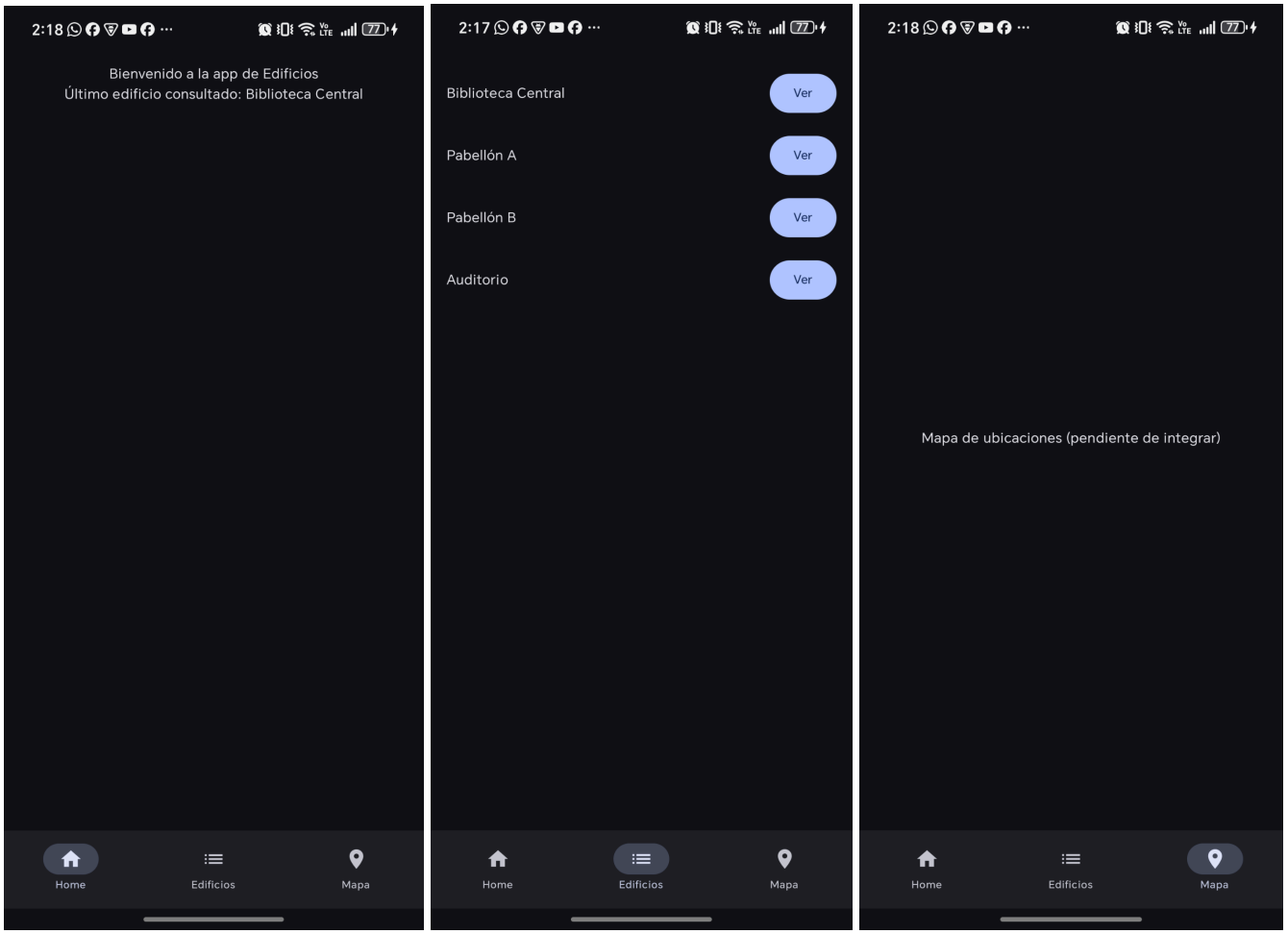
Paso 8: Implementar MapaScreen() con un Text de marcador de posición (por ejemplo "Mapa de ubicaciones (pendiente de integrar)"), y al declarar EdificiosScreen en el NavHost, pasarle una lambda que actualice edificioSeleccionado en el Composable raíz, como se resume en la Figura 2.

```kotlin
// Paso 8: MapaScreen
@Composable
fun MapaScreen() {
    Box(
        modifier = Modifier.fillMaxSize(),
        contentAlignment = Alignment.Center
    ) {
        Text(text = "Mapa de ubicaciones (pendiente de integrar)")
    }
}
```

Paso 9: En MainActivity, dentro del método onCreate(), reemplazar el contenido de setContent { } (el que trae la plantilla por defecto, con Scaffold y Greeting) para que invoque MainScreen() en su lugar.

```kotlin
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        // Paso 9: Reemplazar contenido de setContent
        setContent {
            CampusNavigator_ComposeTheme {
                MainScreen()
            }
        }
    }
}
```

Paso 10: Ejecutar la aplicación en un emulador y verificar que, al pulsar cada ítem de la barra inferior, se muestra la pantalla correspondiente conservando el resaltado del ítem activo, siguiendo la estructura de la Figura 1; luego, seleccionar un edificio en Edificios y confirmar que HomeScreen muestra el valor actualizado al volver a esa pestaña.



2. EJERCICIOS PROPUESTOS|

Tomando como referencia el ejercicio resuelto por el docente, reemplaza la comunicación por lambda entre Edificios y Home por un ViewModel compartido, y agrega una cuarta pestaña:

1. **Crea una clase SeleccionViewModel : ViewModel() con una propiedad mutableStateOf<String> (o StateFlow) para almacenar el edificio seleccionado.**

```kotlin
// 1: ViewModel para almacenar el edificio seleccionado usando StateFlow
class SeleccionViewModel : ViewModel() {
    private val _edificioSeleccionado = MutableStateFlow("Ninguno")
    val edificioSeleccionado: StateFlow<String> = _edificioSeleccionado.asStateFlow()

    fun seleccionarEdificio(nombre: String) {
        _edificioSeleccionado.value = nombre
    }
}
```

2. **Obtén una única instancia de SeleccionViewModel con viewModel() en el Composable raíz y pásala a EdificiosScreen y a HomeScreen (en lugar de la lambda del ejercicio resuelto).**

```kotlin
// 2: Instancia única de SeleccionViewModel en el Composable raíz
@Composable
fun MainScreen(viewModel: SeleccionViewModel = viewModel()) {
        ...
        Scaffold(
        ...
    ) { padding ->
        NavHost(
            ...
        ) {
            // 2: HomeScreen recibe el ViewModel
            composable(Screen.Home.route) {
                HomeScreen(viewModel = viewModel)
            }
            // 2: EdificiosScreen recibe el ViewModel
            composable(Screen.Edificios.route) {
                EdificiosScreen(viewModel = viewModel)
            }
            ...
        }
    }
}
```

3. **En EdificiosScreen, al seleccionar un edificio, actualiza directamente el valor en el ViewModel en vez de invocar una función lambda.**

```kotlin
// 3: EdificiosScreen actualiza el valor directamente en el ViewModel
@Composable
fun EdificiosScreen(viewModel: SeleccionViewModel) {
    val edificios = listOf("Biblioteca Central", "Pabellón A", "Pabellón B", "Auditorio")

    LazyColumn(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp)
    ) {
        items(edificios) { nombre ->
            Row(
                modifier = Modifier
                    .fillMaxWidth()
                    .padding(vertical = 8.dp),
                verticalAlignment = Alignment.CenterVertically
            ) {
                Text(text = nombre, modifier = Modifier.weight(1f))
                Button(onClick = { viewModel.seleccionarEdificio(nombre) }) {
                    Text("Ver")
                }
            }
        }
    }
}
```

4. **En HomeScreen, lee el valor del ViewModel (con collectAsState() si usaste StateFlow) y muéstralo, verificando que el comportamiento visual sea el mismo que con la lambda.**

```kotlin
// 4: HomeScreen observa el StateFlow usando collectAsState()
@Composable
fun HomeScreen(viewModel: SeleccionViewModel) {
    val edificioSeleccionado by viewModel.edificioSeleccionado.collectAsState()

    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(text = "Bienvenido a la app de Edificios")
        Text(text = "Último edificio consultado: $edificioSeleccionado")
    }
}
```

5. Agrega una cuarta pestaña "Perfil" con un Composable simple de marcador de posición, y su NavigationBarItem correspondiente.

Kotlin

```
// 5: Se agrega la cuarta pantalla "Perfil" a la Sealed Class
sealed class Screen(val route: String, val label: String) {
    object Home : Screen("home", "Home")
    object Edificios : Screen("edificios", "Edificios")
    object Mapa : Screen("mapa", "Mapa")
    object Perfil : Screen("perfil", "Perfil")
}

...

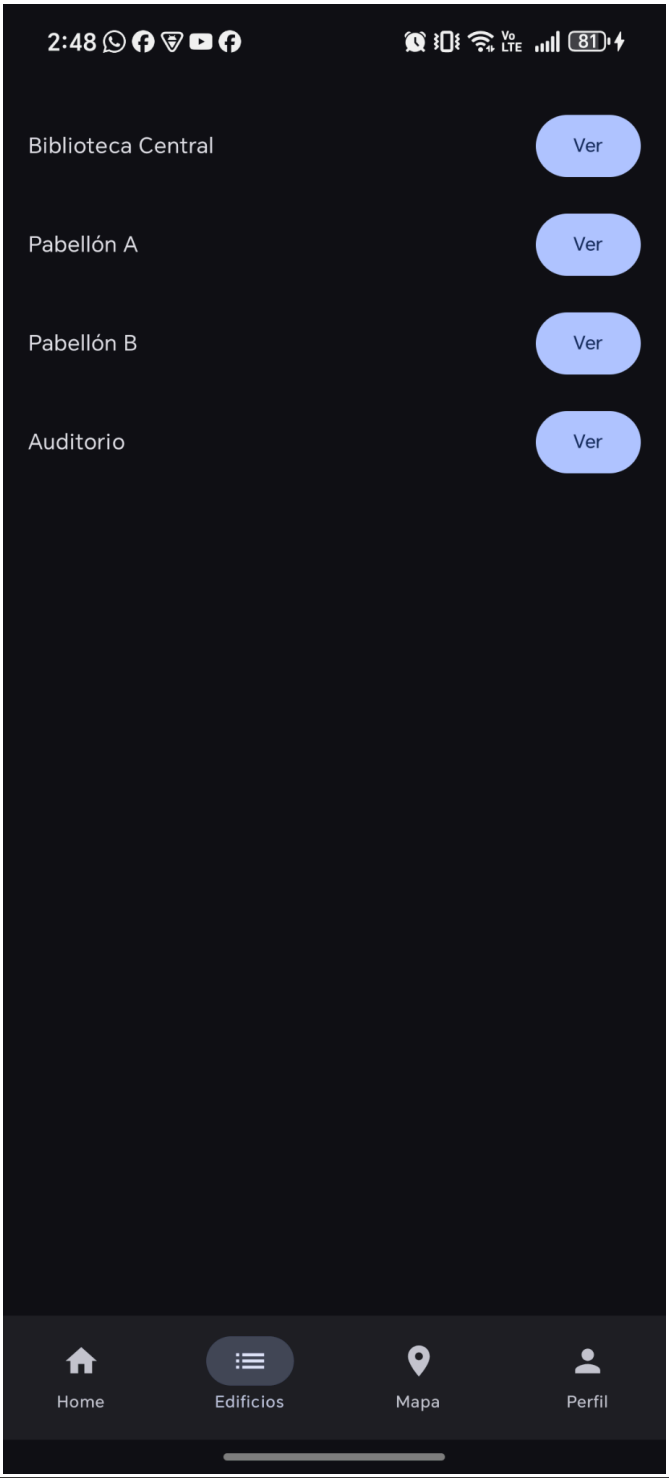
@Composable
fun MainScreen(viewModel: SeleccionViewModel = viewModel()) {
        ...
        Scaffold(
        ...
    ) { padding ->
        NavHost(
            ...
        ) {
            // 5: Composable de la pestaña Perfil
            composable(Screen.Perfil.route) {
                PerfilScreen()
            }
        }
    }
}

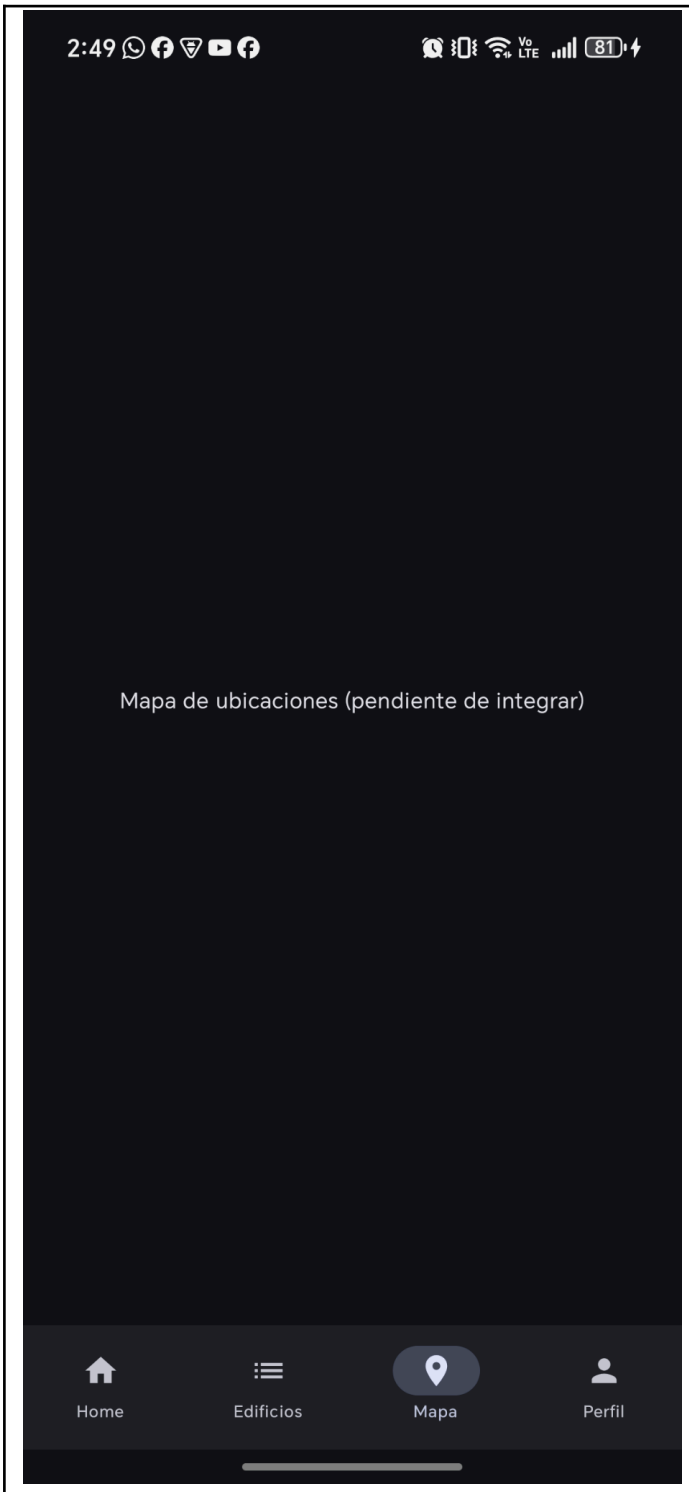
...

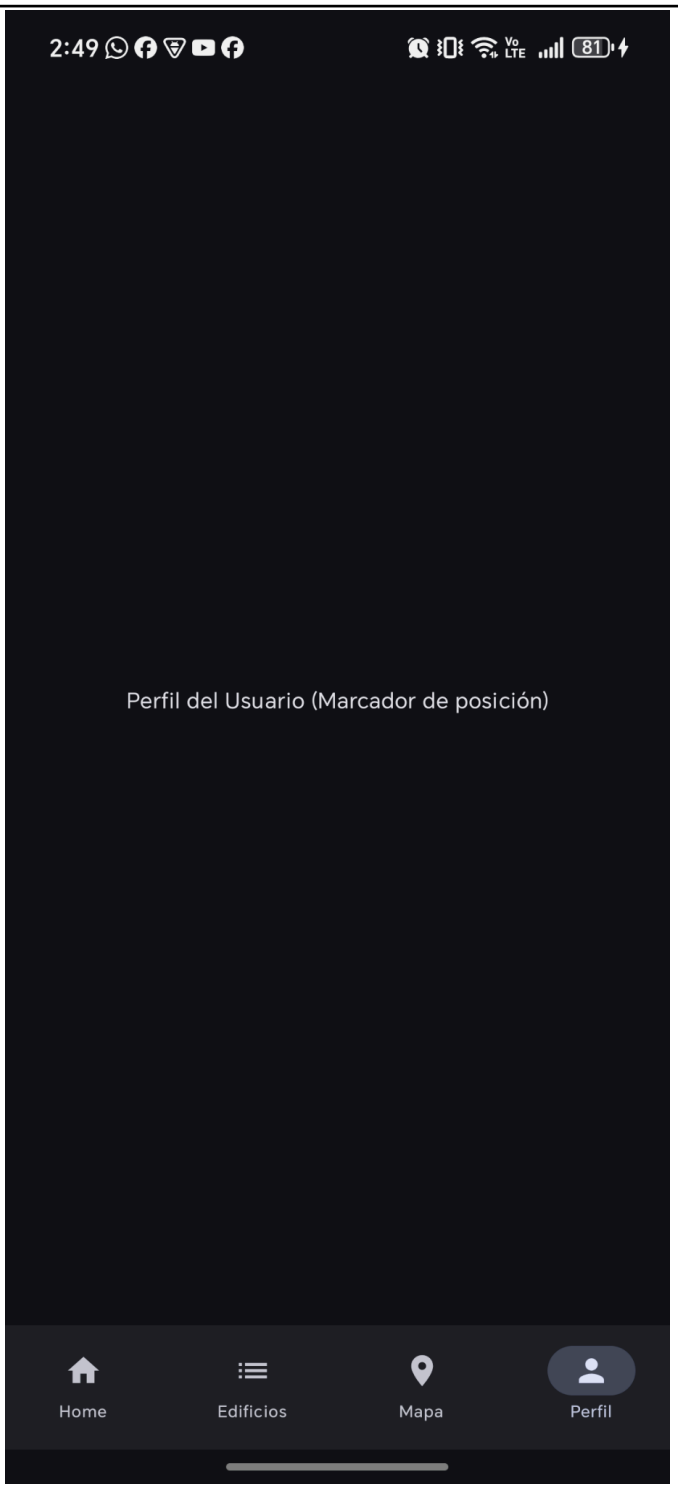
// 5: Pantalla Perfil con marcador de posición simple
@Composable
fun PerfilScreen() {
    Box(
        modifier = Modifier.fillMaxSize(),
        contentAlignment = Alignment.Center
    ) {
        Text(text = "Perfil del Usuario (Marcador de posición)")
    }
}
```

Ejecución de Aplicación:






2:49
Mapa de ubicaciones (pendiente de integrar)
Home
Edificios
Mapa
Perfil


2:49
Perfil del Usuario (Marcador de posición)
Home
Edificios
Mapa
Perfil

CUESTIONARIO:

1. **¿Lograste resolver completamente el Ejercicio/Problema Resuelto por el Docente (pantallas Home, Edificios y Mapa navegables mediante NavigationBar, donde el edificio seleccionado en Edificios se refleja en Home)? Adjunta una captura de pantalla del emulador mostrando el resultado como evidencia.**

Sí, el ejercicio resuelto fue implementado satisfactoriamente en el proyecto CampusNavigator_Compose. Se configuró la navegación mediante NavHost y NavController gestionada a través de un NavigationBar inferior con las rutas "home", "edificios" y "mapa". La comunicación del edificio seleccionado en EdificiosScreen hacia HomeScreen se logró mediante elevación de estado (State Hoisting) en el Composable raíz MainScreen, pasando una función lambda (String) -> Unit como parámetro de evento a EdificiosScreen.

2. **En tu propia implementación, ¿qué diferencia encontraste entre comunicar Composables mediante una función lambda y hacerlo mediante un ViewModel compartido? ¿Cuál de las dos alternativas consideras una mejor práctica? Explica brevemente los motivos.**

Comunicación por Lambdas (State Hoisting): El estado reside en el Composable padre (MainScreen) mediante remember { mutableStateOf(...) }. El valor fluye hacia abajo mediante parámetros y los eventos se notifican hacia arriba mediante funciones lambda. Esto acopla la gestión del estado a la jerarquía directa de la UI.

ViewModel Compartido (SeleccionViewModel): El estado se encapsula dentro del ViewModel utilizando StateFlow. Los Composables leen el estado observando la fuente de verdad (collectAsState()) y modifican el valor invocando directamente funciones del ViewModel, sin depender de la cadena de llamadas de la UI.

Determinación de Mejor Práctica: El ViewModel compartido es la mejor práctica para la gestión de estado entre pantallas navegables.

- Evita el prop drilling (pasar lambdas y estados a través de múltiples niveles de funciones Composable que no utilizan esa información).
- El ViewModel sobrevive a la destrucción y recreación de la Activity (por ejemplo, al rotar la pantalla), reteniendo los datos del usuario. El estado gestionado solo con remember se pierde.
- Separar la lógica de negocio/estado de la capa de presentación facilita la adición de nuevas pantallas o el consumo de APIs/bases de datos en el futuro.

3. **En forma individual, explica qué ventaja adicional ofrece un ViewModel frente a otras formas de mantener el estado al navegar entre pantallas. (Cada integrante del grupo debe incluir su propia respuesta en el informe).**

Al navegar hacia una nueva pantalla con NavHost, la pantalla anterior puede ser destruida o sacada de la jerarquía activa de renderizado. Si el estado depende del árbol de Composables, la información puede corromperse o perderse al regresar. El ViewModel actúa como un repositorio de estado en memoria independiente del árbol de

vistas, permitiendo que múltiples pantallas accedan y modifiquen la misma fuente de verdad sin importar qué pantalla esté en primer plano.

CONCLUSIONES

1. Se comprendió el uso de NavHost, NavController y Sealed Classes junto a NavigationBar para organizar y modularizar eficazmente aplicaciones multipantalla en Jetpack Compose.
2. Se construyó exitosamente la aplicación CampusNavigator_Compose, logrando una navegación fluida entre las pantallas Home, Edificios y Mapa dentro de un contenedor Scaffold.
3. Se concluyó que el uso de un ViewModel compartido con StateFlow es la mejor práctica frente a las lambdas, ya que evita el prop drilling, desacopla la UI de la lógica y preserva el estado ante cambios en el ciclo de vida.

REFERENCIAS Y BIBLIOGRAFÍA

[1] Android Developers, "Navigation with Compose". [En línea]. Disponible: https://developer.android.com/develop/ui/compose/navigation

[2] Android Developers, "ViewModel overview". [En línea]. Disponible: https://developer.android.com/topic/libraries/architecture/viewmodel

[3] Android Developers, "Navigation bar | Jetpack Compose". [En línea]. Disponible: https://developer.android.com/develop/ui/compose/components/navigation-bar

[4] "Cómo navegar entre pantallas en Jetpack Compose con Room y ViewModel (MVVM)", video. [En línea]. Disponible: https://www.youtube.com/watch?v=1ZuBVRYIr7w